

Volume 03

WarrantyGraph

Code Deep-Dive (Annotated)

What the code does and why — real excerpts from the repo

Full project library

This multi-volume set covers the entire repository — not a single session. Algorithms, theory, annotated code, RDF/OWL, pipelines, and deploy are all included.

1. Reliability engine — FMEA ratings

File: graph/reliability.py

```
def severity_rating(severities):
    # WHAT: map symptom labels to FMEA 1-10, take WORST case
    # WHY: safety-critical failures must dominate triage
    ratings = [SEVERITY_SCALE.get(s.lower(), 5) for s in severities]
    return max(ratings) if ratings else 5

def occurrence_rating(evidence_count):
    # WHAT: more closed claims/resolutions → higher 0
    # WHY: empirical prior from field data, not a magic constant
    n = max(int(evidence_count), 0)
    if n == 0: return 3
    ...

def detection_rating(diagnostic_step_count, error_code_count=0):
    # WHAT: more CONFIRMS steps / codes → lower D (easier to detect)
    # WHY: Detection must reflect actual diagnostic assets in the graph
    coverage = step_count + error_code_count
    ...
```

Why pure functions?

No I/O inside reliability.py → unit tests prove determinism without Neo4j (tests/test_reliability.py).

2. Bayesian posteriors

```
def bayesian_posteriors(priors, likelihoods, observed_symptoms,
                        candidate_failure_modes, miss_likelihood=0.15):
    unnormalized = {}
    for fm in candidate_failure_modes:
        score = max(priors.get(fm, 0.0), 0.0)
        for symptom in observed_symptoms:
            # multiply likelihoods; missing edge uses miss_likelihood ≠ 0
            score *= likelihoods.get((symptom, fm), miss_likelihood)
        unnormalized[fm] = score
    total = sum(unnormalized.values())
    return {fm: score/total for fm, score in unnormalized.items()} # normalize
```

Why miss_likelihood=0.15?

If miss=0, one unmatched symptom multiplies the score to zero and kills a good FM. Bounded-away-from-zero likelihoods keep inference stable.

3. Wiring FMEA + Bayes into ranking

File: graph/graph_rag.py — _apply_fmea_and_posteriors

```
def _apply_fmea_and_posteriors(ranked, symptom_ids):
    for row in ranked:
        sev = reliability.severity_rating(row.pop('severities') or [])
        occ = reliability.occurrence_rating(row.pop('evidence_count') or 0)
        det = reliability.detection_rating(row.pop('step_count') or 0)
        row['rpn'] = reliability.rpn(sev, occ, det)
        row['action_priority'] = reliability.action_priority(sev, occ, det)
        row['prior'] = reliability.occurrence_prior(occ)
    # build likelihoods from INDICATES confidences collected in Cypher
    posteriors = reliability.bayesian_posteriors(...)
    # SORT: posterior DESC, then RPN, then link_count
    ranked.sort(key=lambda x: (x['posterior'], x['rpn'], x['link_count']), reverse=True)
```

Why sort by posterior first?

Diagnosis is about which FM is most probable given symptoms. RPN is secondary triage risk, not the primary differential-diagnosis score.

4. Cypher that feeds the scorer

```
MATCH (p:Product {product_id:$pid})-[:CAN_HAVE]->(fm:FailureMode)
OPTIONAL MATCH (s:Symptom)-[:INDICATES]->(fm)
WHERE s.symptom_id IN $symptom_ids
WITH fm, collect(... indications ...), sum(ind.confidence) AS total_confidence,
[ (sv)-[:INDICATES]->(fm) | sv.severity ] AS severities,
size([ (fm)-[:CONFIRMED]->(ev) | ev ]) AS evidence_count,
size([ (ds)-[:CONFIRMS]->(fm) | ds ]) AS step_count
RETURN ...
```

What: Pull candidate FMs for product with observed INDICATES edges and graph-derived S/O/D inputs. **Why:** All scoring inputs come from stored knowledge — not the LLM.

5. Hybrid symptom matching

File: graph/symptom_retrieval.py + match_symptoms

```
def hybrid_symptom_score(user_message, symptom_description, *,
                          lexical_score, corpus=None, lexical_weight=0.45):
    semantic = tfidf_similarity(user_message, symptom_description, corp)
    blended = lexical_weight * lexical_score + (1-lexical_weight) * semantic
    if lexical_score >= 0.3 and semantic >= 0.3:
        blended = max(blended, lexical_score, semantic) # agreement boost
    return min(blended, 1.0)

# In match_symptoms: only score >= min_score count as Bayes evidence;
# weaker matches may still show for UI context (secondary fill).
```

6. diagnose() orchestration

```
def diagnose(user_message, product_id=None, asset_id=None):
    product, asset_ctx, ... = resolve_product_for_diagnosis(...)
    matched_error_codes = match_error_codes(pid, user_message)
    matched_symptoms = match_symptoms(pid, user_message)
    strong_symptom_ids = [s for s in matched if score >= min_score]
    ranked = rank_failure_modes_with_error_codes(...)
    confidence, graph_c, lang_c, rec_strength, dom = _composite_confidence(...)
    # escalation: critical OR (low conf AND (weak language OR ambiguous))
    ambiguous = (top_post - second_post) < diagnosis_ambiguity_margin
    parts = predict_parts(..., fm_posterior=top.posterior)
    return DiagnosisResult(...)
```

Why only strong symptoms for Bayes?

Weak secondary matches are for display. Feeding them into $\text{IIP}(s|fm)$ would invent evidence and create competing FMs the user never really described.

7. Parts predictor

```
SOURCE_RELIABILITY = {
    'REQUIRES_PART': 1.00, 'CLAIM_PRECEDENT': 0.85,
    'BOM_COMPONENT': 0.70, 'SKU_FIT': 0.90,
}
# score = fm_posterior * reliability * edge_probability
base_score = weight * SOURCE_RELIABILITY['REQUIRES_PART'] * (probability or 0.9)
```

8. Shared service + LangGraph + API

```
# services/diagnosis_service.py – single business path
def run_full_diagnosis(...):
    if warranty enriched and not eligible: return warranty_blocked
    result = run_diagnosis(message, product_id, asset_id) # LangGraph
    if escalated and customer+asset: create_case_from_escalation(...)
    return DiagnosisOutcome(...)

# agents/diagnosis_graph.py
detect_product → run_diagnosis → format_response → handle_escalation → END

# api/main.py diagnose()
slot = concurrency_limiter.try_acquire()
message = guard_request(...)
crm = enrich_session_from_crm(...); warranty = check_warranty_eligibility(...)
outcome = run_full_diagnosis(...); validate_output(...)
```

Why diagnosis_service exists?

Historically API and UI could diverge. One function owns warranty + diagnosis + escalation policy.

9. ETL transform snippet

```
# knowledge_etl.py

pairs = parallel_map(connectors.items(), fetch, max_workers=etl_connector_max_workers)
catalog = OntologyBuilder(...).build_catalog_payload(pim, fsm, claims, crm)
# write JSON catalogs + lineage
if load_neo4j:
    populate_graph(driver, catalog)
    invalidate_all_named_caches() # so UI doesn't serve stale subgraphs
```

What: Parallel extract, serial merge, optional graph load. **Why parallel extract:** I/O bound. **Why serial transform:** deterministic confidence merges.

10. Redis rate limit (multi-replica)

```
# guardrails/rate_limit.py - sliding window via Redis ZSET + Lua
# ZREMRANGEBYSCORE old hits; ZCARD; if under max ZADD now; else reject
# WHY: in-process counters diverge across API pods; Redis is shared truth
```

11. File → responsibility cheat sheet

File	What it does	Why it exists
reliability.py	FMEA+Bayes pure math	Deterministic, testable scores
graph_rag.py	End-to-end diagnose	Central intelligence orchestration
symptom_retrieval.py	Hybrid text match	Paraphrase without embeddings infra
parts_predictor.py	Rank parts	Multi-source BOM/claims evidence
diagnostic_engine.py	Tree walk	Guided troubleshooting
populate_graph.py	MERGE catalog + constraints	Idempotent knowledge load + indexes
ontology_builder.py	ETL transform	Canonical catalog from sources
diagnosis_service.py	Business path	One policy for API/UI
diagnosis_graph.py	LangGraph nodes	Explicit multi-step agent
runtime/cache.py	TTL cache	Fast schema/subgraph reads

12. Indexes used by this code (WWWH)

WHAT	Neo4j UNIQUE constraints on every entity *_id (backing unique indexes)
WHERE	graph/populate_graph.py → create_constraints()
WHEN	First lines of every populate_graph / promote / ETL load-to-Neo4j
HOW	CREATE CONSTRAINT IF NOT EXISTS FOR (p:Product) REQUIRE p.product_id IS UNIQUE · ... ×13 labels
WHY	MERGE safety + MATCH {id:\$x} index seeks for diagnose/parts/viz

```
def create_constraints(tx):
    for query in [
        'CREATE CONSTRAINT IF NOT EXISTS FOR (p:Product) REQUIRE p.product_id IS UNIQUE',
        'CREATE CONSTRAINT IF NOT EXISTS FOR (s:Symptom) REQUIRE s.symptom_id IS UNIQUE',
        # FailureMode, Part, Asset, Claim, SKU, Component, ErrorCode, ...
    ]: tx.run(query)

# Called when:
session.execute_write(create_constraints) # then MERGE all nodes
```

WHAT	SQLite PRIMARY KEY + status indexes for agent ops tables
WHERE	utils/persistence.py OperationalStore._init_schema
WHEN	First open of data/diagnostics.db
HOW	PK on case_id/claim_id; idx_escalations_status, idx_claims_status, ...
WHY	List open claims/escalations without full table scan